

Part C – Reproducibility and Workflow

Reproducibility is a core requirement in data science, ensuring that experiments can be validated, extended, and audited by others. In this project, reproducibility was achieved through a combination of Git/GitHub for version control and Data Version Control (DVC) for managing datasets, models, and pipelines. Together, these tools guarantee transparency, collaboration, and end-to-end reproducibility.

1. Git & GitHub

A Git repository was initialised to track all changes to code and documentation:

```
git init
```

To handle large files such as datasets, trained models, and visualisations, Git LFS was configured:

```
git lfs install  
git lfs track "*.csv"  
git lfs track "*.pkl"  
git lfs track "*.png"
```

The repository was then committed and pushed to GitHub for remote backup and collaborative access:

```
git add .  
git commit -m "Initial commit with Git LFS"  
git branch -M main  
git remote add origin  
https://github.com/username/projectname.git  
git push -u origin main
```

A clear README.md file was included to guide users in reproducing results:

```
# Predictive Modelling Project  
  
## Installation  
pip install -r requirements.txt  
  
## Running the Project  
python src/main.py  
  
## Expected Results  
- Regression metrics: RMSE, MAE
```

- *Classification metrics: Accuracy, F1 score, Confusion Matrix*
 - *Visualisations stored in /results*
-

2. Data Version Control (DVC)

While Git manages code effectively, DVC was employed to track data and models, enabling reproducibility of large artefacts and structured workflows.

Initialisation

```
dvc init  
git commit -m "Initialise DVC"
```

Tracking Datasets and Models

```
dvc add data/dataset.csv  
dvc add models/model.pkl  
git add data/dataset.csv.dvc models/model.pkl.dvc .gitignore  
git commit -m "Track dataset and model with DVC"
```

Pipeline Automation

A reproducible pipeline was created in dvc.yaml, linking preprocessing, feature engineering, modelling, and evaluation:

```
dvc stage add -n preprocess \  
-d src/preprocess.py -d data/raw.csv \  
-o data/clean.csv \  
python src/preprocess.py  
  
dvc stage add -n features \  
-d src/features.py -d data/clean.csv \  
-o data/features.csv \  
python src/features.py  
  
dvc stage add -n train \  
-d src/train.py -d data/features.csv \  
-o models/model.pkl \  
python src/train.py  
  
dvc stage add -n evaluate \  
-d src/evaluate.py -d models/model.pkl \  
-o results/metrics.json \  
python src/evaluate.py
```

Reproducing Results

dvc repro

This command rebuilds the workflow end-to-end, guaranteeing that results are always consistent and reproducible across environments.

Conclusion

By combining GitHub for version control, collaboration, and documentation with DVC for data management and pipeline automation, the project achieves full reproducibility. This ensures that results are:

- Transparent – every step is documented and tracked,
- Auditable – data and model versions are preserved, and
- Replicable – results can be regenerated exactly with `dvc repro`.

This integrated approach aligns with best practice in both academic research and professional machine learning projects, ensuring the robustness and credibility of the findings.

Final Github Link:

<https://github.com/yuot86/assignment1-dsts>